

MGS636 Lab 4

Building a Local RAG Architecture

Homework Goals: While base generative AI models are typically great at “general-purpose” activities, sometimes we might wish we had a specialized tool for our use cases. Traditionally, this meant that you would need to retrain the model (or part of it) with your own dataset to introduce new information. However, specifically with LLMs, there is a unique way to handle knowledge augmentation efficiently and without additional training. This method, called Retrieval Augmented Generation (RAG), leverages the unique nature of the LLMs context window and the storage capabilities of traditional databases to process your query to the LLM, **retrieve** relevant documents from a database, **augment** the prompts context window with the additional information, and **generate** a response with the new context. This allows an LLM to leverage a knowledge base that extends far beyond its base training dataset.

In this assignment, we will build two RAG architectures. One using Open WebUI and one we will build from scratch in Python—both using Ollama as a back end for our LLM.

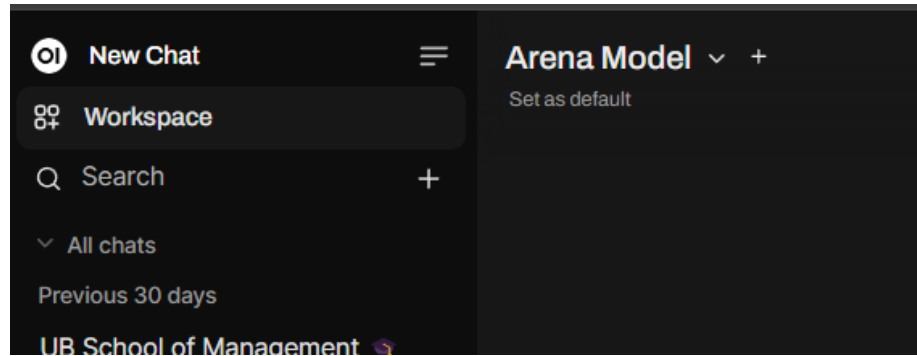
Deliverables:

At the end of the assignment, you will deliver the following:

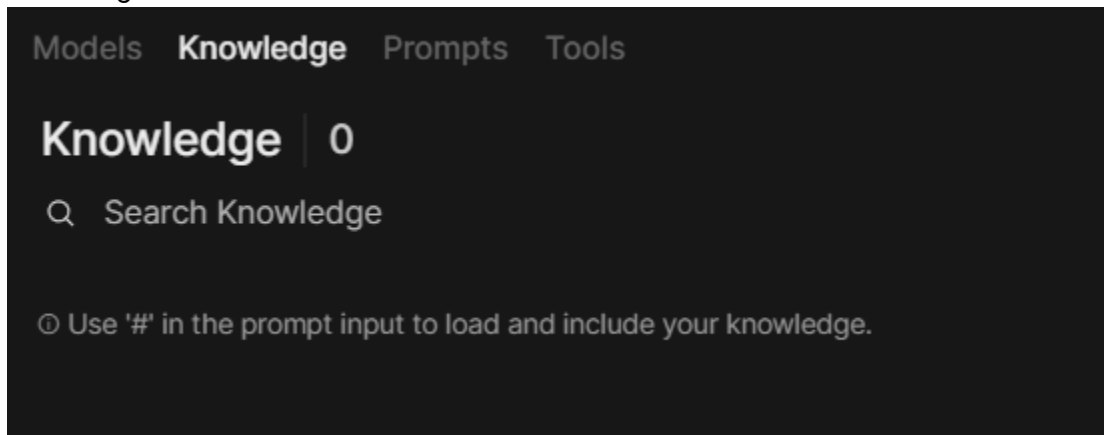
1. A 1-page summary of your learnings and experiences.
2. Screenshots as designated throughout the assignment (highlighted in yellow).
3. Answers to any questions posed throughout the assignment (highlighted in yellow).
4. A screen recording (Recommended to use Zoom and share a link) showing your local custom coded RAG architecture in operation (highlighted in yellow).

Building a simple RAG using Open WebUI

1. Open up Open WebUI (and make sure Ollama is currently running on your computer). If you don't remember how to open up Open WebUI, refer to the steps in homework 2, which detail how to access the virtual environment you created in Python for the software. Remember that Open WebUI will typically run at <http://localhost:8080>
2. Log in, if prompted. If you don't remember your username and password, you can follow this guide to reset it: <https://docs.openwebui.com/troubleshooting/password-reset/>
 - a. Alternatively, if you find these steps too daunting, I might recommend deleting the folder structure for Open WebUI and reinstalling it following the steps in homework 2.
3. In the top left of the screen, you will see a menu, click it (if it's not already expanded) and select “Workspace”



4. Once in workspace, you will see some options on the top of the screen. Select "Knowledge" and then click the "+" to the right side of the screen to create a new knowledge database.



Note how it says “use # in the prompt input to load and include your knowledge.” This is critical.

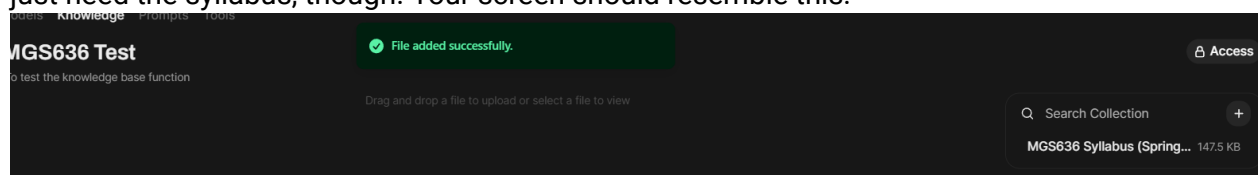
5. When creating a new knowledge base, use the following parameters:

A screenshot of the 'Create a knowledge base' form. It has two text input fields. The first is labeled 'What are you working on?' and contains the text 'MGS636 Test'. The second is labeled 'What are you trying to achieve?' and contains the text 'To test the knowledge base function'. Below these fields is a 'Visibility' section with a 'Public' dropdown (showing a lock icon) and the text 'Accessible to all users'. At the bottom right is a 'Create Knowledge' button.

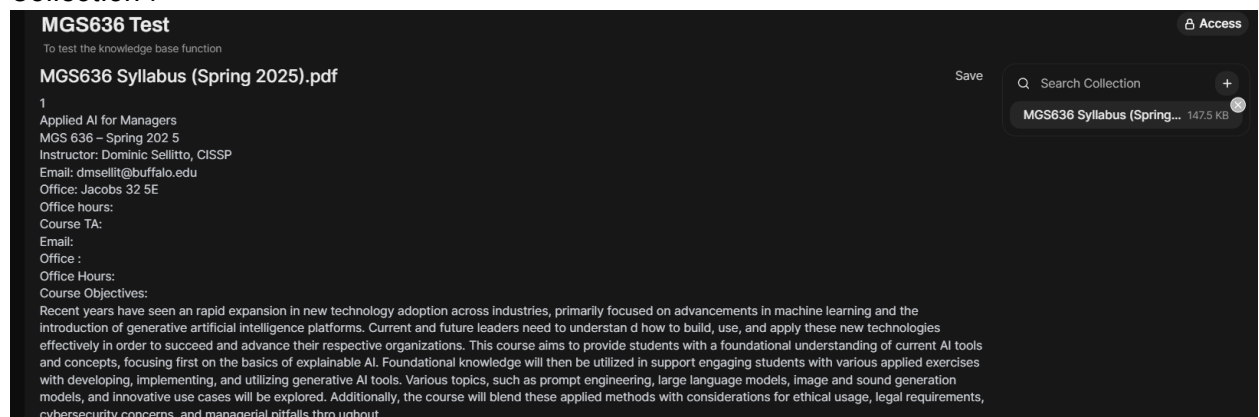
Note that “public” does not mean open to anybody—public just means anybody else using your instance of Open WebUI would have access. Remember, this is all fully local and running on your computer.

Click “Create Knowledge”.

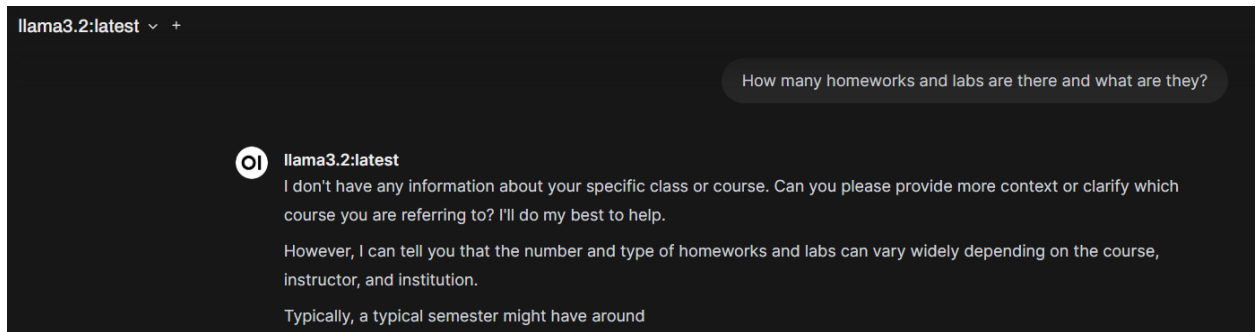
- Now, I want you to download the syllabus for this class from UBLearns. Once it is downloaded, you can take it and drag it into the Open WebUI window for this knowledge base. You can also click the “+” on the right side of the screen to see this and additional options. One function that is particularly useful is the option to upload an entire directory. Often, your knowledge base will be large. It can be tedious to upload documents one by one. This allows you to simply upload an entire folder. For now, we just need the syllabus, though. Your screen should resemble this:



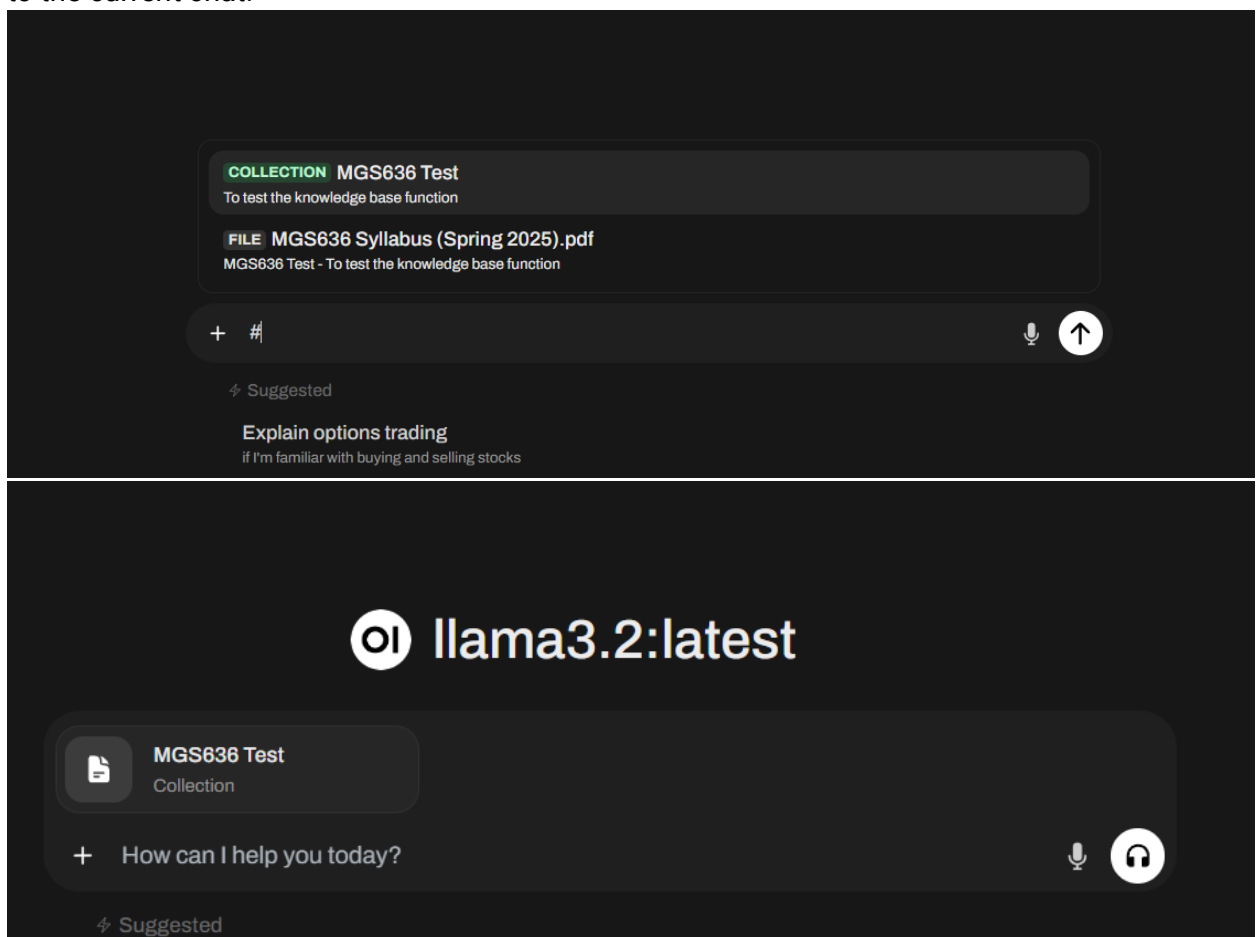
- To view the contents of an uploaded file, you can click on the file name under “Search Collection”:



- Now, let's try to query the syllabus. Click on “New Chat” in the top left.
- In the new chat, select one of the models you previously downloaded for Ollama. I would recommend a smaller model, but you can choose any one you wish. Do not select arena model, as that is meant for scoring multiple models against each other by not telling you which model generated an output.
- Ask the following question: “What topics does this class cover?” Paste a screenshot of your prompt and response in your report. Here's what I get:



11. Now, let's try invoking our knowledge base. Create a new chat. In the chat input, start by typing "#". You will see the collection pop up as an option. Click on it to add it as context to the current chat.



12. Now, type the same question as before: "What topics does this class cover?" **Paste a screenshot of your results.**
13. **Write 3-5 sentences detailing the differences between the first prompt and the second with the collection added. Consider answering some of the following... What was different in the responses? How about the performance? Why do you think these changes happened between the two prompts? What other things does Open WebUI provide you with when you query a knowledge collection?**

Trying it yourself

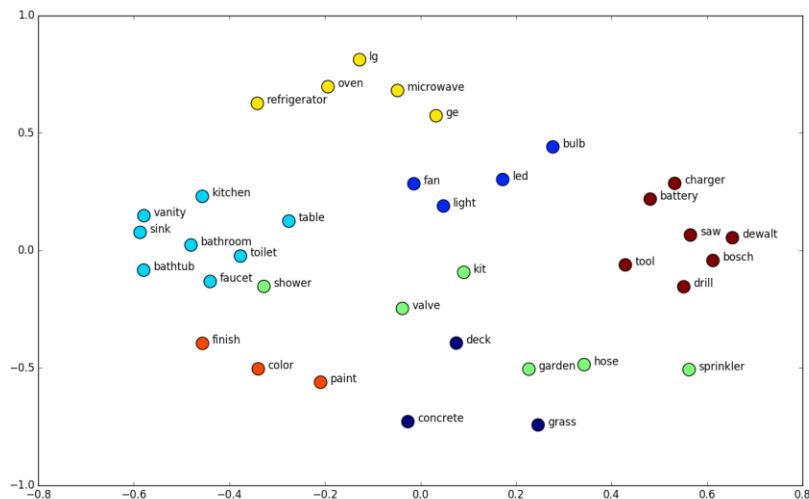
1. Create a new knowledge collection with some documents of your choosing. These can be documents of yours, from the Internet, etc. Choose at least 3 documents (I recommend no more than 5 depending on your computer hardware, as more documents may reduce the speed). Paste a screenshot of your created knowledge base in your report.
2. Write a query to an LLM of your choice based on the documents, trying both a query without the knowledge collection and with the knowledge collection included to identify the differences. Paste a screenshot of your outputs from each query.
3. Try the same query with the knowledge collection but a different LLM (one of the ones you downloaded for HW2). Paste a screenshot of the output.

Creating your own RAG tool with Python

While using a tool like Open WebUI is great for one-off use cases, it is not necessarily “production-ready,” meaning you might need more flexibility if you are wanting to include RAG as part of an enterprise, consumer-facing application. In these cases, we may wish to build our own RAG architecture. In doing so, we can learn quite a bit more about how RAG works, and how to tune a RAG to meet our needs. We need three key parts in order to create a RAG from scratch: An interface, an LLM inference engine, and a vector database. We already have our inference engine (Ollama), so that really leaves us with the interface (which is going to be very simple) and our vector database. Let’s start with the vector database first.

What is a vector database?

This is the location where we will store all of our documents. You might be wondering why we don’t just use an SQL database. We *could*, but vector databases offer a multitude of advantages over SQL for most use cases of RAG architectures. The biggest one is in the name “vector”. See, when we run our text input query to the LLM in a RAG architecture, first our program will conduct a lookup on the document database (like our knowledge collection from Open WebUI). In an SQL database, this would amount to effectively a keyword search. If I search for “What’s the textbook for the class” it will look for the words in the query—like textbook—and search them in the document set. This is great, and may even work in some instances, but it lacks one very important thing: **context**. Semantic context can allow us to try to match not only keywords with the documents, but the meanings of words. For example, if you were to ask “what do I have to read for the class?” and the syllabus only contained the word “textbook”, in a traditional keyword lookup you would return zero results. But, vectorized data (using embedding models) captures words and their relationships, allowing the tools to base results on similarity in terms of context as opposed to simple keywords. This allows for less precise inputs to still produce valuable outputs, even where you don’t get the words *exactly* correct. It’s also helpful when you just want to find things that are generally related to other things—for example, “Which homeworks will work on programming” will semantically relate “Python” to programming because they exist near similar vectors.



Example of word embedding

We learn more about this in class, so I won't dive too much deeper, but essentially what we're going to do here is use an **existing** embedding model (there are many free ones out there) to create word vectors out of our documents for the RAG. Then, what we store in the vector database is those documents and their vectors. When an input query is sent, we run that query through the same embedding model, then look for the most similar documents in the knowledge base, returning the most similar results.

Let's start!

1. Open the IDLE application you installed with Python. **Note:** if you are familiar with other IDE's (visual studio, sublime text, etc.) you may absolutely use those instead of IDLE.
2. First, we need to install some dependencies. We'll use Python's pip function to do so. The primary package we will need is called "llama-index," which is a very popular Python library for engaging with LLMs. It contains a number of useful functions we will need for our ecosystem. **Open a command prompt (terminal)** and type "py -m pip install llama-index" without the quotes. If py doesn't work, you will need to use the keyword your operating system has for python. This could be "py3" "python" or "python3". I'll be referring to it as "py" throughout the assignment, so please replace with your specific use case.
 - Note: While these instructions tell you to run the pip install outside of a virtual environment, you may consider creating a new virtual environment as described in homework 2. This can be valuable if you are developing multiple applications to ensure packages don't conflict with one another. You'll note that I will be completing this exercise in a virtual environment.
3. We're also going to need to install chromadb, which we will use for our vectorstore database. To do so, type "py -m pip install chromadb".
 - By the way, here are some details on each of these libraries:
 - i. <https://www.trychroma.com/>

ii. <https://docs.llamaindex.ai/en/stable/>

4. Once llama-index and chromadb installed, we can begin writing our code. Back in IDLE, click “File” and then “New File”. Immediately save this untitled file as “ragtest.py”. **Very important:** don’t forget where you saved this. We’ll need it for later. If you have created a virtual environment, save this file in that virtual environment to ensure ease of access.
 - One other thing worth noting: if you have created a virtual environment, you can only use that virtual environment command prompt to run the script.
5. If you aren’t familiar with Python, the first thing we need to do is to import some useful functions from our libraries. **Why doesn’t it load all packages and functions?** For efficiency! Us specifying only the libraries needed helps ensure only the minimum packages are loaded. We’ll also use this step to confirm everything installed properly.
6. Back in IDLE, paste the code below to import the functions. I’ve added a print(“success”) at the end. As Python executes line-by-line, it will attempt to import all of our libraries and only reach the “print” function if successful. I’ll talk about why we’re importing these as we go.

```
import chromadb
from llama_index.core import PromptTemplate, Settings, SimpleDirectoryReader,
StorageContext, VectorStoreIndex
from llama_index.core.node_parser import SentenceSplitter
from llama_index.embeddings.huggingface import HuggingFaceEmbedding
from llama_index.llms.ollama import Ollama
from llama_index.vector_stores.chroma import ChromaVectorStore

print('success')
```

What are these libraries? These libraries span two tools: ChromaDB and llamaindex. You’ll see how they are going to be used in our code in a bit, but you can probably guess how we’re going to use some of them already. For example: llama_index.llms.ollama is going to allow us to query our Ollama inference server. llama_index.embeddings.huggingface will allow us to create word embeddings based on the documents we’re going to put into our RAG.

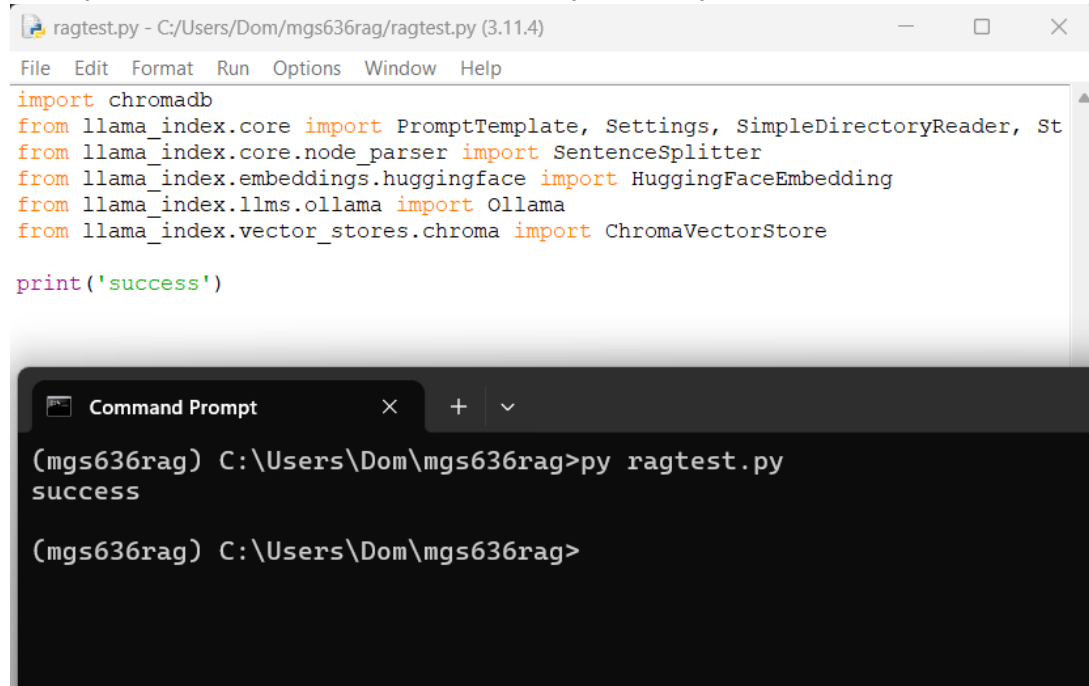
7. You may receive an error, particularly in lines 5, 6, and 7 talking about missing modules. If you receive this error, you can resolve them one by one by manually installing the missing dependencies:
 - pip install llama-index-embeddings-huggingface
 - pip install llama-index-llms-ollama
 - pip install llama-index-vector-stores-chroma
8. It is also possible you will get a dependency error, saying that your tokenizers version doesn’t match either transformers or chromadb. **ONLY IF YOU GET AN ERROR**, this can be resolved by typing the following (run each line independently one at a time):

```
pip install chromadb==0.5.23
pip uninstall transformers
```

```
pip uninstall tokenizers
pip install transformers==4.46.1
pip install tokenizers==0.20.3
```

Why would you have to do this? With so many different libraries, certain functions depend on other functions. With how fast the AI software is advancing, certain libraries are only compatible with other libraries even at specific versions. Here, we're specifying those version (0.5.23, for example).

9. Once you get the dependencies sorted out, you'll see your success message print:

The image shows a screenshot of a Python script in a text editor and its execution in a command prompt. The script, named ragtest.py, is located at C:/Users/Dom/mgs636rag/ragtest.py (Python 3.11.4). The script imports chromadb and various components from llama_index (core, node_parser, embeddings, llms, vector_stores). It then prints a success message. Below the script, a Command Prompt window shows the command 'py ragtest.py' being executed, resulting in the output 'success'.

10. Now, let's replace our print function with an API call to Ollama (a query to our LLM inference server) to make sure that is all working. Remove "print('success')" from your code in IDLE and replace it with the following:

```
llm = Ollama(model="llama3.2")
response = llm.complete("What is the university at buffalo?")
print(response)
```

Don't forget to save.

In this code, we're creating a new variable (llm) to store the information about contacting our llama3.2 instance hosted by Ollama. If you want to use a different LLM, you can replace llama3.2 with the name you use to run it (you can find this by running *ollama list* like we did in homework 2. Then, we run the llm.complete function with a static question. This will query our llm and store the output in the response variable. Then, we just print the response! Run the code. **Paste a screenshot of the output in the command prompt.**

Congratulations, you *technically* just created an LLM-powered application.

11. Replace the query with 3 other questions and paste the results as screenshots into your report.

12. Change the LLM to one of the other Ollama LLMs you downloaded during HW2. Type the same question "What is the university at buffalo?" and **paste your results**.

Coding our RAG

13. To begin, we need to create a client and a collection for our ChromaDB vector database, where we will store our RAG documents. The client allows us to interact with ChromaDB functions, while the collection is essentially our "database" for the RAG. First, we need to delete (or comment out) the following lines of code we added earlier:

```
response = llm.complete("What is the university at buffalo?")
print(response)
```

Leave the "llm = Ollama" line of code.

14. Next, we need to add the following lines of code:

```
chroma_client = chromadb.EphemeralClient()
chroma_collection = chroma_client.create_collection("mgs636test")
```

The first line will create a ChromaDB server and a client for you to interact with it, all only running while the system is running. The second line uses that existing chromadb client to create a new database ("collection") called mgs636test.

15. Next, we need to set a word embedding model to create the word embeddings for our chromadb collection based on our input files. There are a lot of these available online, but the one we are going to use for this exercise is called "bge-small-en" from hugging face. More information can be found here:

<https://docs.llamaindex.ai/en/stable/examples/embeddings/huggingface/>

For now, you can add the following code to download the embedding model and instantiate it as the "embed_model" variable. This may take some time to download:

```
embed_model = HuggingFaceEmbedding(model_name="BAAI/bge-small-en")
```

16. By now, your code should look like this:

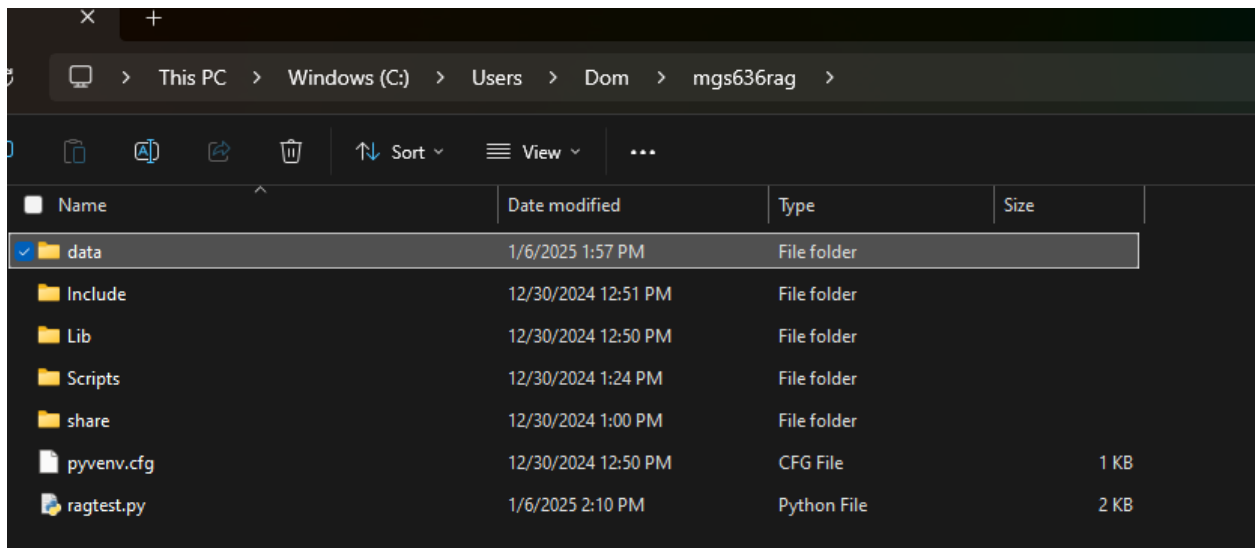
```
import chromadb
from llama_index.core import PromptTemplate, Settings, SimpleDirectoryReader, StorageContext, VectorStoreIndex
from llama_index.core.node_parser import SentenceSplitter
from llama_index.embeddings.huggingface import HuggingFaceEmbedding
from llama_index.llms.ollama import Ollama
from llama_index.vector_stores.chroma import ChromaVectorStore

llm = Ollama(model="llama3.2")

chroma_client = chromadb.EphemeralClient()
chroma_collection = chroma_client.create_collection("mgs636test")

embed_model = HuggingFaceEmbedding(model_name="BAAI/bge-small-en")
```

17. Take the MGS636 Syllabus document you downloaded (in PDF format). Create a folder in **the same location you have your mgs636ragtest.py file**. Call it "data". This is where we are going to store our documents for the LLM to reference in the RAG. Put the PDF syllabus in the "data" folder.



18. Now, we're ready to write our RAG code. In order to reference the embed models and llm's, we need to set the "Settings" parameter for llamaindex. This is a library we imported earlier. Currently you have a line in your code that says:

- `llm=Ollama(model="llama3.2")`

You are going to *change this line so it reads:*

- `llm=None`

Why? Llamaindex, by default, will reference OpenAI (ChatGPT) and this avoids situations where it might look for an API key and throw an error. This isn't strictly required, so you can remove the entire line if you wish and see how it behaves.

So, you might be wondering where we tell it what llm we're going to use, right? That's where "Settings" comes into play. Under the llm line, you are going to add the following:

- `Settings.llm=Ollama(model="llama3.2", request_timeout=360.0)`

Basically we're telling llamaindex "anywhere that we don't specifically specify an LLM, you should use this as the LLM." But, now we've added a component here called "request_timeout". This is because our query might take some time, and what llamaindex does is send a *http request* (standard web request) to our Ollama server. If we don't specify a long request_timeout period, the request may fail and say "hey, we never got a reply." We're setting this timeout to a value that should be able to fit properly. If later on you get a "request timeout" error, you can always increase that value beyond 360.

19. We also need to change our embed_model settings like we did with the LLM. Below the "embed_model = " line from earlier, add the following:

- `Settings.embed_model = embed_model`

As the code would imply, we're telling it to set the `embed_model` setting to the same value as the `embed_model` variable. You might be wondering "can't we just do that with the LLM instead of setting it to `None`?" And, the answer is technically yes. I encourage you to test your code out at the end and try to optimize it. At any rate, your current code should look like this:

```
import chromadb
from llama_index.core import PromptTemplate, Settings, SimpleDirectoryReader, StorageContext, VectorStoreIndex
from llama_index.core.node_parser import SentenceSplitter
from llama_index.embeddings.huggingface import HuggingFaceEmbedding
from llama_index.llms.ollama import Ollama
from llama_index.vector_stores.chroma import ChromaVectorStore

llm= None

Settings.llm=Ollama(model="llama3.2", request_timeout=360.0)

chroma_client = chromadb.EphemeralClient()
chroma_collection = chroma_client.create_collection("mgs636test")

embed_model = HuggingFaceEmbedding(model_name="BAAI/bge-small-en")
Settings.embed_model=embed_model
```

It's super important that our "`Setting.embed_model=embed_model`" comes after we have the `embed_model` variable set in the line above. This is because Python interpreters read line-by-line and won't allow you to set something equal to a variable that doesn't exist yet (or, rather, hasn't been "instantiated").

If you're not comfortable with Python, I would recommend running this code after you make these modifications. In the current state (as long as yours looks like mine above) it won't actually do anything (we haven't told it to), but you can quickly identify errors this way.

20. Next, we need to create another variable to store our RAG documents. We're going to, unsurprisingly, call this variable "documents" and put a new line in our code with the following:

- `documents = SimpleDirectoryReader("./data/").load_data()`

Remember earlier, we put the PDF file for the syllabus in a folder called `data`? This uses the library "Simple Directory Reader" to "Load Data" from the `data` folder (`./` refers to the current directory, this is why it is important to put the `data` folder where your Python script is—this is called a "relative path". You can also set this to an "absolute path" by writing out the entire pathway to the file. This would look like "`C:/Users/Dom/MGS636/Ragtest/data/`". We add the `/` at the end so that it's known that we want every file in that folder.

21. Now, we can write the code to create our vector store database. This will use the "ChromaVectorStore" library. First, we instantiate a `vector_store` variable tied to our `mgs636test` collection we created earlier:

- `vector_store = ChromaVectorStore(chroma_collection=chroma_collection)`

22. Next, we use the StorageContext library

(https://docs.llamaindex.ai/en/stable/api_reference/storage/storage_context/) we imported to tie the ChromaDB vector store to llamaindex:

- `storage_context = StorageContext.from_defaults(vector_store=vector_store)`

23. Finally, we create a VectorStoreIndex, which will build the actual object that our query will reference to pull out the most relevant information

(https://docs.llamaindex.ai/en/stable/module_guides/indexing/vector_store_index/).

This line of code pulls our documents, storage_context, and embed_model together to create the index:

- `index = VectorStoreIndex.from_documents(documents, storage_context=storage_context, embed_model=embed_model)`

Now, your code will look like this:

```
import chromadb
from llama_index.core import PromptTemplate, Settings, SimpleDirectoryReader, StorageContext, VectorStoreIndex
from llama_index.core.node_parser import SentenceSplitter
from llama_index.embeddings.huggingface import HuggingFaceEmbedding
from llama_index.llms.ollama import Ollama
from llama_index.vector_stores.chroma import ChromaVectorStore

llm= None

Settings.llm=Ollama(model="llama3.2", request_timeout=360.0)

chroma_client = chromadb.EphemeralClient()
chroma_collection = chroma_client.create_collection("mgs636test")

embed_model = HuggingFaceEmbedding(model_name="BAAI/bge-small-en")

Settings.embed_model=embed_model

documents = SimpleDirectoryReader("./data/").load_data()

vector_store = ChromaVectorStore(chroma_collection=chroma_collection)
storage_context = StorageContext.from_defaults(vector_store=vector_store)
index = VectorStoreIndex.from_documents(documents, storage_context=storage_context, embed_model=embed_model)
```

24. We're almost there. Now, we just need to set our query engine. This is the basic interaction method with our LLM, that allows for more flexibility than the "llm.complete" function we were testing earlier. In this case, our query engine will use our index (which we set up in step 23) to pull information, then execute it together in a query to the LLM and return a full response. Here's the line of code we'll use:

- `query_engine = index.as_query_engine(llm=Settings.llm)`

25. Then, we need to set up our prompt to use that query engine. We're going to set it as a static prompt in code, but I'll show you how to do a dynamic prompt a bit later. Here's the code:

- `response = query_engine.query("What topics are covered in this class?")`

26. Finally, we need to print our response. This is pretty simple:

- `print(response)`

Your code should look like this:

```

import chromadb
from llama_index.core import PromptTemplate, Settings, SimpleDirectoryReader, StorageContext, VectorStoreIndex
from llama_index.core.node_parser import SentenceSplitter
from llama_index.embeddings.huggingface import HuggingFaceEmbedding
from llama_index.llms.ollama import Ollama
from llama_index.vector_stores.chroma import ChromaVectorStore

llm= None

Settings.llm=Ollama(model="llama3.2", request_timeout=360.0)

chroma_client = chromadb.EphemeralClient()
chroma_collection = chroma_client.create_collection("mgs636test")

embed_model = HuggingFaceEmbedding(model_name="BAAI/bge-small-en")

Settings.embed_model=embed_model

documents = SimpleDirectoryReader("./data/").load_data()

vector_store = ChromaVectorStore(chroma_collection=chroma_collection)
storage_context = StorageContext.from_defaults(vector_store=vector_store)
index = VectorStoreIndex.from_documents(documents, storage_context=storage_context, embed_model=embed_model)

query_engine = index.as_query_engine(llm=Settings.llm)
response = query_engine.query("What topics are covered in this class?")
print(response)

```

27. Run the code. It will take some time to run on your laptop, maybe even a few minutes. This is okay and expected. Once it runs, paste a screenshot of the command prompt with the results alongside your code in your IDE (like IDLE) in your report.

Congratulations, you've built a simple RAG using ollama and llamaindex! It's surprisingly not that many lines of code, right?

Well, this is just a *simple* RAG pipeline. There are a number of things we're missing:

- Actually *saving* the vector database and index—right now, it's just creating a new one in real time (that's the definition of "ephemeral" in the ChromaDB ephemeral client). If we want to query this system often, it is very inefficient to do things this way.
- Control over our prompt—what is the prompt? It's more than just our question.
- Hyperparameter tuning of the RAG... how do we control how many documents are reference in a query (top_k)?

Modifying your RAG system

28. This is where you may need to step outside of your comfort zone a bit further. So far, we've been following an example that isn't too dissimilar to the llamaindex sample RAG system for ChromaDB (https://docs.llamaindex.ai/en/stable/examples/vector_stores/ChromaIndexDemo/). But, as I said above, there's a problem. We're basically compiling the vectors every single time we run the script. This is very inefficient. We want to compile the vectors once, then use those vectors later on. For this, we would need to switch from the ChromaDB ephemeral client to the persistent client. This means that we'll be storing a copy of our

ChromaDB outputs to our computer so we can leverage it later. This time, however, I want you to try to do this on your own by modifying your code. You'll have to change some things. You can use the code here as a guide for what you will need to modify: <https://docs.llamaindex.ai/en/stable/understanding/storing/storing/#using-vector-stores>

HINTS:

- Despite what the documentation says, you actually will need to import another library. Be sure to add the following line:
 - **from chromadb import PersistentClient**
- Remember what I said above about an ephemeral client? If we want a persistent database, it would make sense we might need a persistent client.
- You will need to create a new folder, called "chroma_db" in the folder where you have your Python script (same as you did with the data folder)
- You will only need to modify the following lines of code, but will need to reference the link above in order to do so:
 - **chroma_client = chromadb.EphemeralClient()**
 - In the documentation, llamaindex refers to the chroma_client variable as "db"
 - **chroma_collection = chroma_client.create_collection("mgs636test")**
 - In the documentation, llamaindex uses the "quickstart" collection instead of our "mgs636test"
- Follow the example on the Llamaindex website

When complete, run the script to make sure it works. Once you are successful, paste a screenshot of your code along with a screenshot of the output from the file and another screenshot with the contents of the chroma_db folder on your computer. You should see a few files created with your chromadb database.

Great! Now let's resave our file here under a new name. The new name for this file will be "ragcreate.py" as opposed to the original "ragtest.py" because this script will now be used whenever we want to create a new RAG database. We don't want to use it when we want to *run* our rag system, because it has all of those extra "document load" functions that will slow things down needlessly (after all, why load a document when we already have it in our persistent vector store database?).

29. Next, we're going to create a copy of this file (however you choose) and call it "ragrun.py". Open that in IDLE (or your IDE of choice).

30. Back on the llamaindex demo page (linked in step 28), if you scroll a bit past the original demo code, you'll see additional code you can use if you have already created and stored your embeddings.

If you've already created and stored your embeddings, you'll want to load them directly without loading your documents or creating a new VectorStoreIndex:

```
import chromadb
from llama_index.core import VectorStoreIndex
from llama_index.vector_stores.chroma import ChromaVectorStore
from llama_index.core import StorageContext

# initialize client
db = chromadb.PersistentClient(path="./chroma_db")

# get collection
chroma_collection = db.get_or_create_collection("quickstart")

# assign chroma as the vector_store to the context
vector_store = ChromaVectorStore(chroma_collection=chroma_collection)
```

Use this code to modify your "ragrun.py" file to make it so it is fit for running an existing RAG datastore.

HINTS:

- You don't need any additional libraries
- You'll need to remove the line pertaining to your simple directory reader
- You'll need to change your "index=" function to match what llamaindex provides

When complete, run the ragrun.py script and make sure it works. **Once you are successful, paste a screenshot of your code along with a screenshot of the response in the command prompt.**

As a sidenote... you might not notice a huge speed increase here. That's because we have a pretty simple RAG with only one file in it. The benefit of storing your RAG database for later is that the performance benefit will scale with the number of documents you store in that RAG.

Great! You now have a persistent way to create a RAG database and a way to run it.

Building your own RAG

31. Now I want you to try this on your own. Build your own custom RAG database using whatever documents you wish. These can be web documents or other types of documents. Make the RAG persistent, and query it. **Attach your code along with screenshots of your process for building the RAG and a short video screenshare of your**

RAG system running (I recommend using Zoom to screen record. Voiceover is not necessary).

32. Here are some additional criteria I want you to consider:

- Look up how to change the top-k and integrate it in your code
 - i. Hint: the function is called “similarity_top_k” and you can put it into your existing code as part of your query_engine.query function.
- Look up how to change your prompt using the “PromptTemplate” library built in to llamaindex. Attempt to modify the default prompt to something of your own.

It is okay if you are unable to do these things—but I expect each student to try. If you are successful, provide a screenshot of your code and of the output of the queries run. If you are not successful, document the steps you took to try to resolve the issue (e.g., web searches, asking another AI how to fix it, etc.). I will provide the solutions and code after the assignment is due.

Bonus point: You will be eligible for one bonus point if you identify the one library we imported into our code but did not use at all during this assignment, and if you do a bit of research and tell me how we may use that library (5-7 sentences).